

# Advanced Computer Architecture

## Homework 3

Federico Vassallo 416376

### Kernel 1

```
1 void kernel1 (int array[ARRAY_SIZE]) {
2     #pragma HLS array_partition variable=array type=cyclic factor=6 dim=1
3     loop: for(int i = 0; i < ARRAY_SIZE; i++) {
4         #pragma HLS unroll factor=2 skip_exit_check=false
5         #pragma HLS pipeline II=1 rewind=false style=stp
6         array[i] = (array[i] << 2) + array[i];
7     }
8 }
```

Figure 1: Modified code for Kernel 1.

### Discussion

The first optimization was changing the code from the multiplication `array[i] = array[i] * 5;` to the following line: `array[i] = (array[i] << 2) + array[i];`. Since  $5 = 4 + 1 = 2^2 + 1$ , the operation is equivalent. Shifting by a constant is simply wiring in hardware so it has no latency, replacing the 4-cycle multiplication with a single 1-cycle addition.

The second optimization was pipelining the loop with `#pragma HLS pipeline II=1 style=stp`. Each iteration is independent of the others (since `array[i]` only depends on its own old value, there is no loop-carried dependency). I use `style=stp` since `frp` and `flp` bring no benefit here.

The third optimization was loop unrolling. I have chosen an unroll factor of 2 to keep the partition factor (and so the number of BRAM banks) at a reasonable number. Larger unroll factors would get a better performance but at the cost of more adders and significantly more BRAM banks.

The fourth optimization was array partitioning, which is required to support both the pipelining and the unrolling. With `unroll factor=2` and `II = 1`, four memory operations happen in parallel every cycle: two loads from the current iterations for example (`array[4]` and `array[5]`) and two stores from the iterations that started two cycles earlier so (`array[0]` and `array[1]`). Since each BRAM has only a single port, these four accesses must be in four distinct banks. the minimum cyclic partition factor that avoids bank conflicts is 6, giving the pragma

```
#pragma HLS array_partition variable=array type=cyclic factor=6 dim=1.
```

I use `cyclic` because consecutive indices are accessed simultaneously.

## Scheduling Dataflow Graph

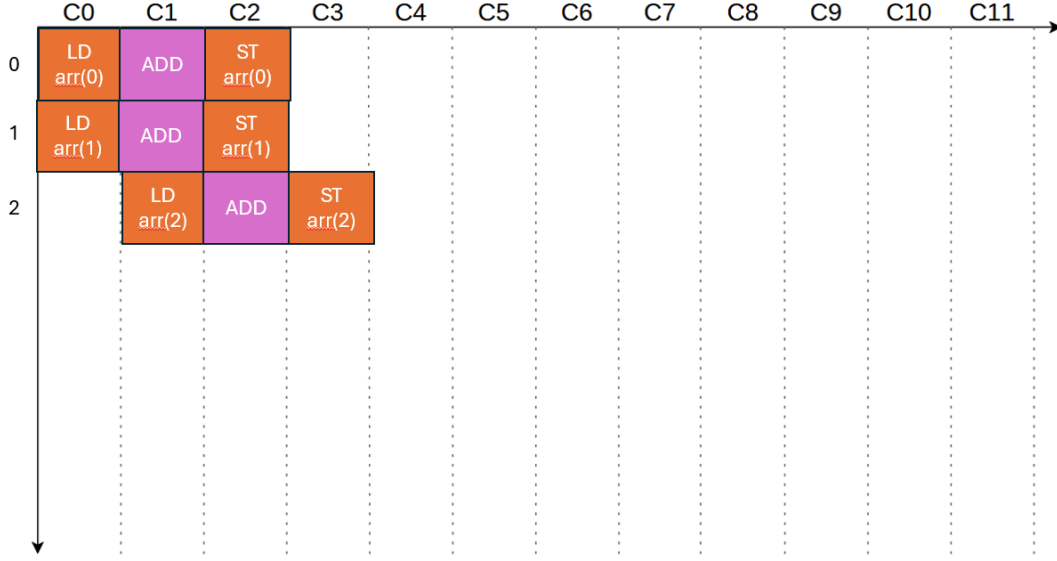


Figure 2: Scheduling dataflow graph for the first three iterations.

Iterations 0 and 1 are processed in parallel as the two elements of pass 0 (unroll factor 2); iteration 2 belongs to pass 1, which starts one cycle later ( $II = 1$ ).

## Cycle Count

| Quantity      | Number of cycles    |
|---------------|---------------------|
| One iteration | 3                   |
| Entire task   | $ARRAY\_SIZE/2 + 2$ |

## Resources Required

| Resources Required |             |                                        |
|--------------------|-------------|----------------------------------------|
| Adders             | Multipliers | Ports (array)                          |
| 2                  | 0           | 6 read/write ports, cyclic partitioned |

## Trade-off

The total latency drops from a baseline of roughly  $4 \times ARRAY\_SIZE$  cycles (the original loop limited by the 4-cycle integer multiplier) to  $LATENCY = ARRAY\_SIZE/2 + 2$  cycles, an approximately 8x improvement. The transformation of the multiplication to a shift and an addition enable to replace the 4-cycle integer multiplier with a 1-cycle adder. unrolling by factor 2 contributes further.

A more aggressive design could push further: unroll factor 4 with partition factor 12 or fully unrolling the loop would give a constant 3-cycle latency independent of  $ARRAY\_SIZE$ . These options would require, respectively, 4 adders and 12 BRAM banks, or  $ARRAY\_SIZE$  adders and  $ARRAY\_SIZE$  BRAM banks, so a cost that scales linearly with the unroll factor. Unroll factor 2 with partition factor 6 provides a balance between latency and resource use.

## Kernel 2

```
1 void kernel2 (int array[ARRAY_SIZE]) {
2     int previous0 = array[0];
3     int previous1 = array[1];
4     int previous2 = array[2];
5
6     loop: for(int i = 3; i < ARRAY_SIZE; i++) {
7         #pragma HLS pipeline II=3 rewind=false style=stp
8         int result = previous2 + previous1 * previous0;
9         array[i] = result;
10
11         previous0 = previous1;
12         previous1 = previous2;
13         previous2 = result;
14     }
15 }
```

Figure 3: Modified code for Kernel 2.

## Discussion

The kernel contains a loop-carried dependency: each iteration writes `array[i]` and the next iteration reads it back as `array[(i+1)-1]`.

The first optimization is to add scalar registers. The original code reads the three previous values from memory each iteration. We instead keep them in three scalar registers `previous0`, `previous1`, `previous2`. They are initialized once before the loop, that is the reason why I didn't put the first cycle of LD `array[0]`, `array[1]` and `array[2]` in the dataflow graph since that is done before the loop. The registers rotate them after each iteration. This removes all three loads from the loop body and the next iteration no longer has to wait for the current iteration's store to commit to BRAM.

The second optimization is pipeline the loop with `#pragma HLS pipeline II=3 style=stp`. We cannot do better than `II=3` due to dependencies, as shown in the picture below:

|             |                                                   |
|-------------|---------------------------------------------------|
| ITERATION 0 | <u>array[3]</u> = array[2] + array[1] · array[0]; |
| ITERATION 1 | array[4] = <u>array[3]</u> + array[2] · array[1]; |
| ITERATION 2 | array[5] = array[4] + <u>array[3]</u> · array[2]; |

The result of the `array[3]` is needed at both for the addition of the iteration 1 and the multiplication of the iteration 2. In particular this dependency for the multiplication of the iteration 2 is what forces to use the `II = 3`

Array partitioning is unnecessary because the loop body now contains no LDs and only one ST per iteration a single-port BRAM is sufficient.

A possible further optimization is manual unrolling by a factor of 2. This can reduce the average cost from 3 cycles per original iteration to 2.5 cycles per original iteration. However, this requires

duplicate arithmetic units (2 multipliers, 2 adders), a more complex code, and cleanup logic if `ARRAY_SIZE` is odd. The single-unrolled version has a balance between latency and complexity. Array partitioning brings no benefit here because the loop body contains no BRAM reads and only one BRAM write per iteration; a single-port BRAM is already sufficient.

### Scheduling Dataflow Graph

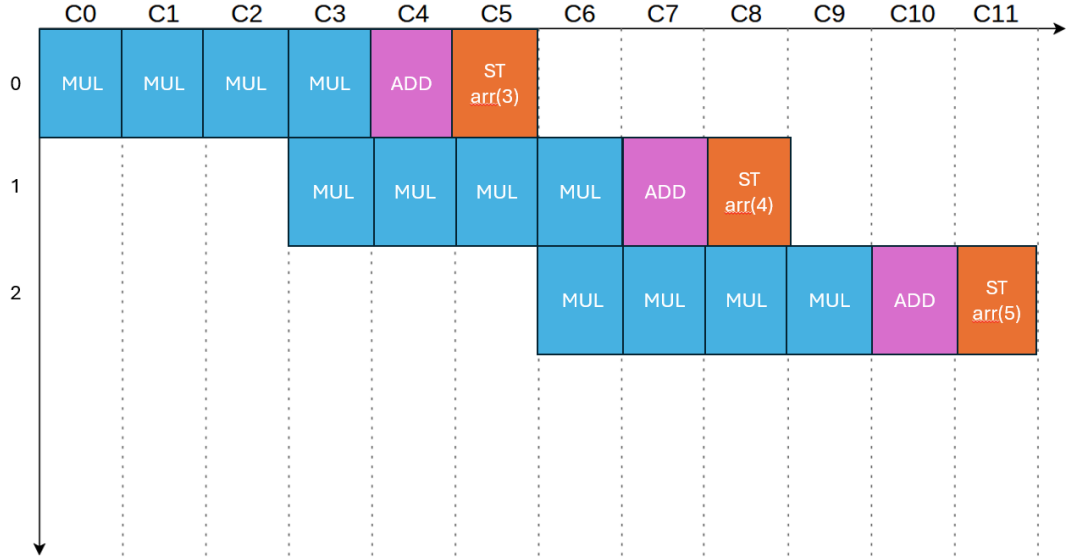


Figure 4: Scheduling dataflow graph for the first three iterations.

### Cycle Count

Because of pipelining, a new iteration starts every 3 cycles. The loop runs from  $i = 3$  to  $i = \text{ARRAY\_SIZE} - 1$ , so there are  $\text{ARRAY\_SIZE} - 3$  iterations that means that the  $\text{LATENCY} = 3(\text{ARRAY\_SIZE} - 3) + 3 = 3 * \text{ARRAY\_SIZE} - 6$ .

| Quantity      | Number of cycles             |
|---------------|------------------------------|
| One iteration | 6                            |
| Entire task   | $3 * \text{ARRAY\_SIZE} - 6$ |

### Resources Required

| Resources Required |             |                   |                 |
|--------------------|-------------|-------------------|-----------------|
| Adders             | Multipliers | Ports (array)     | Extra registers |
| 1                  | 1           | 1 read/write port | 3               |

## Kernel 3

```
1 void kernel3(float hist[ARRAY_SIZE], float weight[ARRAY_SIZE],
2             int index[ARRAY_SIZE]) {
3     static float partial_hist[5][ARRAY_SIZE] = {0};
4     #pragma HLS array_partition variable=partial_hist type=complete dim=1
5     #pragma HLS array_partition variable=hist type=cyclic factor=3 dim=1
6
7     int current_line = 0;
8     main_loop: for (int i = 0; i < ARRAY_SIZE; i++) {
9         #pragma HLS pipeline II=1 rewind=false style=stp
10        partial_hist[current_line][index[i]] =
11            partial_hist[current_line][index[i]] + weight[i];
12        current_line = (current_line == 4) ? 0 : current_line + 1;
13    }
14
15    merge_loop: for (int k = 0; k < ARRAY_SIZE; k++) {
16        #pragma HLS pipeline II=1 rewind=false style=stp
17        float original_hist = hist[k];
18        float line0 = partial_hist[0][k];
19        float line1 = partial_hist[1][k];
20        float line2 = partial_hist[2][k];
21        float line3 = partial_hist[3][k];
22        float line4 = partial_hist[4][k];
23
24        float p0 = original_hist + line0;
25        float p1 = line1 + line2;
26        float p2 = line3 + line4;
27        float q0 = p0 + p1;
28        float result = q0 + p2;
29
30        hist[k] = result;
31    }
32 }
```

Figure 5: Modified code for Kernel 3.

## Discussion

The original loop `hist[index[i]] = hist[index[i]] + weight[i]` has a 5 cycle dependency chain through the BRAM: load `hist[idx]` (1 cycle), float add (3 cycles), store back (1 cycle). Since static HLS cannot determine at compile time whether `index[i]` and `index[i+1]` alias, it must assume they do and serialize the chain, forcing `II = 5` in the original loop that brings total of  $\approx 5N$  cycles.

The optimization exploits the associativity of float addition: the contributions to each histogram bin can be summed in any order, as long as every `weight[i]` eventually lands in `hist[index[i]]`. Reordering floating-point additions can produce results that differ from the sequential order in the last few mantissa bits, so the optimized kernel produces an approximation of the original output rather than a bitwise identical result. I did it since it is allowed for the purposes of this homework.

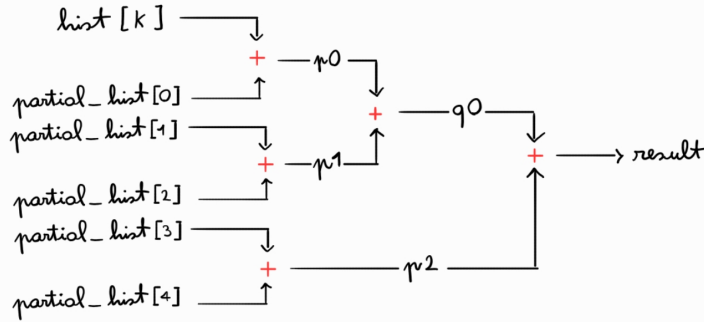
To exploit the associativity I maintain  $F = 5$  independent partial histograms (`partial_hist[5][ARRAY_SIZE]`) and round-robin updates across them: iteration  $i$  writes to `partial_hist[i mod 5]`. Consecutive iterations now touch different physical BRAMs and have no dependency on each other; iterations  $i$  and  $i + 5$  share the same shadow and are exactly 5 cycles apart in real time matching

the dependency chain length. In this way the main loop pipelines at  $\Pi = 1$ .

The choice of  $F = 5$  is because it equals the dependency chain length, which is the smallest  $F$  that lets the main loop reach  $\Pi = 1$ . The shadows are partitioned with `type=complete dim=1`, splitting them into 5 independent single-port BRAMs to make parallel access possible.

The `current_line` variable is the rotating shadow selector. It increases by 1 until it gets to 4 and so it restarts to 0.

Because of this I had to add a final merge loop to combine the 5 partial histograms back into `hist[]`. For each  $k$ , the merge reads `hist[k]` and `partial_hist[0..4][k]` in parallel since they are in 6 separate BRAM, sums them through an adder tree, and writes the result back to `hist[k]`. The structure of the adder tree is shown in the figure below



The merge phase has its own bottleneck: `hist[]` is single ported. Within one merge iteration, the read of `hist[k]` happens at cycle 0 and the write at cycle 10 (after the 9-cycle add tree and the 1-cycle store), giving a 10-cycle read-write gap. Under  $\Pi = 1$ , iteration  $j$ 's write at real cycle  $j + 10$  would collide with iteration  $j + 10$ 's read at the same real cycle.

To achieve  $\Pi = 1$  in the merge as well, I cyclically partition `hist` with factor 3:

```
#pragma HLS array_partition variable=hist type=cyclic factor=3 dim=1
```

This splits `hist[]` into three banks so that: bank A holds `hist[0]`, `hist[3]`, `hist[6]`, ..., bank B holds `hist[1]`, `hist[4]`, ..., bank C holds `hist[2]`, `hist[5]`, .... The schedule of the merge loop, showing how the three banks alternate accesses, is given in the figure below .

| CYCLE | BANK A                                | BANK B                                | BANK C                                |
|-------|---------------------------------------|---------------------------------------|---------------------------------------|
| 0     | ITER 0<br>READ <code>hist[0]</code>   |                                       |                                       |
| 1     |                                       | ITER 1<br>READ <code>hist[1]</code>   |                                       |
| ⋮     | ⋮                                     | ⋮                                     | ⋮                                     |
| 10    | ITER 0<br>WRITE <code>hist[0]</code>  | ITER 10<br>READ <code>hist[10]</code> |                                       |
| 11    |                                       | ITER 1<br>WRITE <code>hist[1]</code>  | ITER 11<br>READ <code>hist[11]</code> |
| 12    | ITER 12<br>READ <code>hist[12]</code> |                                       | ITER 2<br>WRITE <code>hist[2]</code>  |
| 13    | ITER 3<br>WRITE <code>hist[3]</code>  | ITER 13<br>READ <code>hist[13]</code> |                                       |

The condition for no port conflict is that  $F$  must not divide the 10-cycle read-write gap.  $F = 3$  is the smallest factor satisfying this constraint.

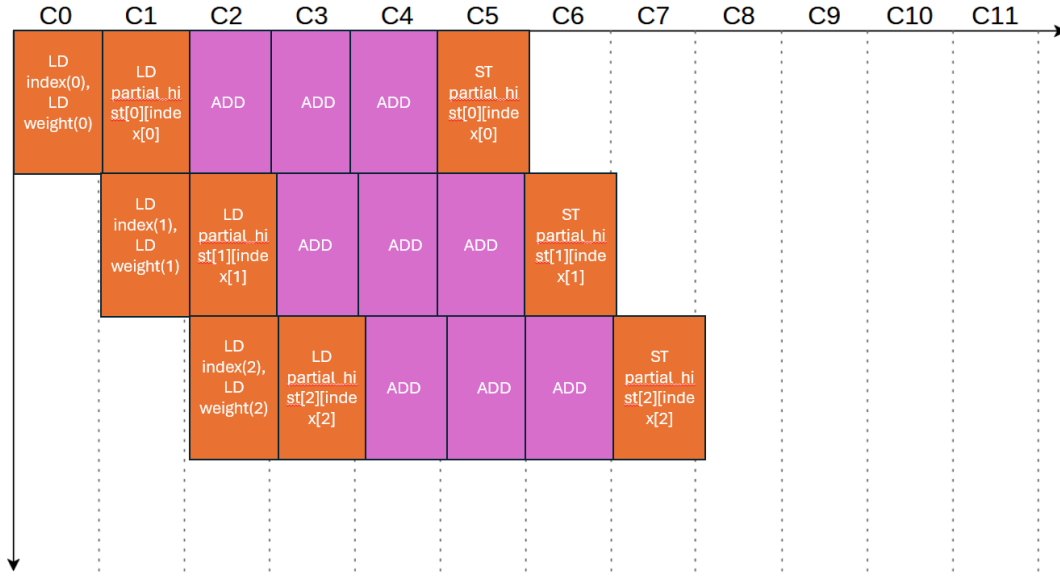


Figure 6: Scheduling dataflow graph of the main\_loop for the first three iterations

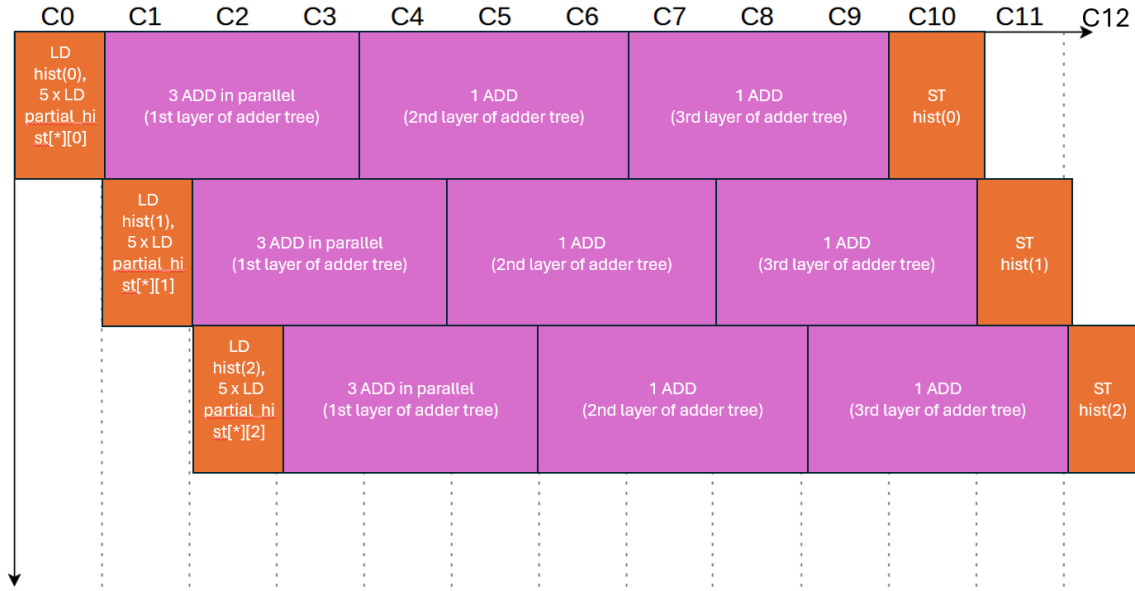


Figure 7: Scheduling dataflow graph of the merge\_loop for the first three iteration

## Cycle Count

Both loops pipeline at  $\Pi = 1$ . The main loop has a latency of 6 cycles per iteration. The merge loop has a latency of 11 cycles per iteration (1 LD for the parallel reads of `hist[k]` and the five shadows, 9 cycles for the 3-level float-add tree (each level is a 3-cycle float add) and 1 ST).

| Quantity                 | Number of cycles                  |
|--------------------------|-----------------------------------|
| One main-loop iteration  | 6                                 |
| One merge-loop iteration | 11                                |
| Main loop total          | $\text{ARRAY\_SIZE} + 5$          |
| Merge loop total         | $\text{ARRAY\_SIZE} + 10$         |
| Entire task              | $2 \cdot \text{ARRAY\_SIZE} + 15$ |

## Resources Required

| Resources Required  |             |                                                                                                                             |                                   |
|---------------------|-------------|-----------------------------------------------------------------------------------------------------------------------------|-----------------------------------|
| Adders              | Multipliers | BRAMs (data)                                                                                                                | Extra registers                   |
| $1 + 5 = 6$ (float) | 0           | $3$ ( <code>hist</code> ) + $5$ ( <code>partial_hist</code> )<br>+ $1$ ( <code>weight</code> ) + $1$ ( <code>index</code> ) | $1$ ( <code>current_line</code> ) |

The float adder count is 1 adder in the main loop and 5 adders in the merge phase forming the add tree. The dominant memory cost is the  $5\times$  replication of histogram-sized arrays via `partial_hist`, plus the  $3\times$  partitioning of `hist`. The merge loop needs 5 floating-point adders because, with  $\text{II} = 1$ , three iterations are simultaneously in the pipeline at different tree layer of the add tree: three layer 1 adders, one layer 2 adder, and one layer 3 adder all in the same cycle. At cycle 7, for instance, iteration 6 is doing the three layer 1 adds, iteration 3 is doing the 1 layer 2 add, and iteration 0 is doing the one layer 3 add.

## Trade-off

The optimized implementation reduces the total latency from a baseline of  $\approx 5 \cdot \text{ARRAY\_SIZE}$  cycles (the original loop limited to  $\text{II} = 5$  by the load-fadd-store chain through `hist[]`) to  $2 \cdot \text{ARRAY\_SIZE} + 15$  cycles so an approximately 2.5x speedup. The hardware price is 5 additional histogram-sized BRAMs (the partial histograms) plus 2 additional BRAMs to support the factor-3 partition of `hist`. Float adders increase from 1 to 6, but no multipliers are needed. The reordering of floating-point additions during the merge phase may produce results that differ from the original code in the last few mantissa bits due to rounding effects.

## Kernel 4

```
1 void kernel4(int array[ARRAY_SIZE], int index[ARRAY_SIZE], int offset) {
2
3     int result = array[offset];
4     int currentarray = array[offset + 1];
5
6     loop: for (int i = offset + 1; i < ARRAY_SIZE - 1; ++i) {
7         #pragma HLS pipeline II=1 rewind=false style=stp
8         int nextarray = array[i + 1];
9         result = result + index[i] * (nextarray - currentarray);
10        currentarray = nextarray;
11    }
12
13    array[offset] = result;
14 }
```

Figure 8: Modified code for Kernel 4.

## Discussion

The first optimization I did is a simplification of the original expression. The original loop had

$$\text{array}[\text{offset}] = \text{array}[\text{offset}] - \text{index}[i] \cdot \text{array}[i] + \text{index}[i] \cdot \text{array}[i+1].$$

This can be rewritten as

$$\text{array}[\text{offset}] = \text{array}[\text{offset}] + \text{index}[i] \cdot (\text{array}[i+1] - \text{array}[i]).$$

Now each iteration requires only one multiplication instead of two. Since multiplication has a latency of 4 cycles, the performance is improved.

The second optimization is to keep the value of `array[offset]` in the scalar variable `result`. In the original code, `array[offset]` is read and written at every iteration, even though it is always the same memory location. We can use an intermediate value that is updated in a register and written back to memory only once at the end. This removes the loop-carried memory dependence on `array[offset]` and reduces the number of accesses to the `array` BRAM.

The third optimization is to reuse consecutive values of `array`. In iteration `i`, the computation needs both `array[i]` and `array[i+1]`. However, `array[i+1]` becomes `array[i]` in the next iteration. For this reason, I store the previous value in `currentarray` and only load the new value into `nextarray`. This reduces the loop body to only one new load from `array` per iteration. I assume that the loop executes at least one iteration:

$$\text{offset} + 1 < \text{ARRAY\_SIZE} - 1$$

With this assumption, the initialization loads `array[offset]` and `array[offset+1]` are always valid. So I assumed the number of iterations to be strictly larger than  $N = 0$ .

Then, I apply loop pipelining with

```
#pragma HLS pipeline II=1 style=stp.
```

After the previous transformations, each iteration performs one read from `index`, one read from `array`, one subtraction, one multiplication, and one addition. Since `array` and `index` are two different arrays, each mapped to a single-port BRAM, one read per array per cycle is sufficient. So, array partitioning is not required.

An initiation interval of 1 is possible also because the loop-carried dependency through `result` does not force a larger II, because the addition using `result` happens only after the multiplication, giving enough time for the previous value of `result` to be produced.

The downside of this optimization is that the code is less direct than the original one and uses extra scalar registers. However, it significantly reduces the number of memory accesses and multiplications per iteration.

## Scheduling Dataflow Graph

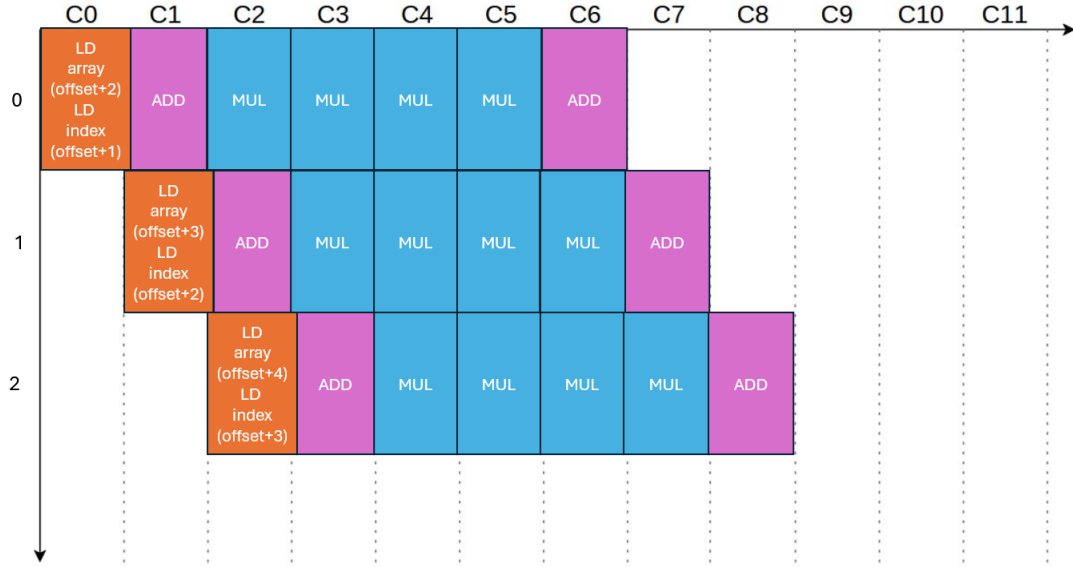


Figure 9: Scheduling dataflow graph for the first three iterations.

## Cycle Count

The loop runs from  $i = \text{offset} + 1$  to  $i = \text{ARRAY\_SIZE} - 2$ , so it executes  $\text{ARRAY\_SIZE} - \text{offset} - 2$  iterations. Each iteration has a latency of 7 cycles, and with  $\text{II} = 1$  a new iteration starts every cycle. The pipelined loop therefore takes

$$(\text{ARRAY\_SIZE} - \text{offset} - 2 - 1) \cdot 1 + 7 = \text{ARRAY\_SIZE} - \text{offset} + 4$$

cycles. The final store of `array[offset]` adds 1 more cycle, giving a total of  $\text{ARRAY\_SIZE} - \text{offset} + 5$  cycles.

| Quantity      | Number of cycles                         |
|---------------|------------------------------------------|
| One iteration | 7                                        |
| Entire task   | $\text{ARRAY\_SIZE} - \text{offset} + 5$ |

## Resources Required

| Resources Required |             |                              |                              |                 |
|--------------------|-------------|------------------------------|------------------------------|-----------------|
| Add/Sub units      | Multipliers | Ports ( <code>array</code> ) | Ports ( <code>index</code> ) | Extra registers |
| 2                  | 1           | 1 read/write port            | 1 read port                  | 3               |

## Trade-off

The reaches  $II = 1$ , so it does one element per cycle. To go faster we could unroll the loop. The loop is a cumulative sum: every iteration adds into the same **result**, so the iterations depend on each other. If we just unrolled and kept one accumulator, the additions would still happen one after another and we would gain nothing.

The solution could be to give each unrolled iteration its own partial sum, let them add up in parallel, and then do one final addition at the end to combine them. With an unroll factor of 2 this processes two elements per cycle and roughly halves the total number of cycles. The price is more hardware: a second multiplier, more adders, and we also have to partition **array** and **index** with **cyclic factor=2**, because each step now reads two neighbouring elements at once. An issue would be that the number of iterations depends on **offset** and can be odd, so there would be the need to handle a leftover iteration.

Also, since the loop is just a cumulative sum, we could even unroll it completely: all the products are independent, so they could all be computed at once and then summed with an adder tree. This would be very fast, but extremely expensive, we would need one multiplier per iteration and would have to read the whole array in a single cycle, which means storing every element in its own register. For a realistic array size this would not be worth it.

## Kernel 5

```
1  float kernel5(float bound, float a[ARRAY_SIZE], float b[ARRAY_SIZE]) {
2      #pragma HLS array_partition variable=a type=cyclic factor=2 dim=1
3      #pragma HLS array_partition variable=b type=cyclic factor=2 dim=1
4
5      float result = 0.0f;
6      bool found = (bound <= 0.0f);
7
8      loop: for (int i = 0; i + 1 < ARRAY_SIZE; i += 2) {
9          #pragma HLS pipeline II=1 rewind=false style=stp
10
11         float sum1 = a[i]      + b[i];
12         float sum2 = a[i + 1] + b[i + 1];
13
14         bool crossed1 = (sum1 >= bound);
15         bool crossed2 = (sum2 >= bound);
16
17         if (!found) {
18             result = crossed1 ? sum1 : sum2;
19             found = crossed1 || crossed2;
20         }
21     }
22
23     if ((ARRAY_SIZE % 2) == 1 && !found) {
24         result = a[ARRAY_SIZE - 1] + b[ARRAY_SIZE - 1];
25     }
26
27     return result;
28 }
```

Figure 10: Modified code for Kernel 5.

## Discussion

The original implementation used a `while` loop with a runtime-dependent exit condition:

```
while (sum < bound && i < ARRAY_SIZE)
```

So the number of executed iterations depends on the input data. Such behavior is difficult for a Static HLS scheduler because the compiler cannot know at compile time how many iterations will execute.

To improve the performance I changed the kernel to a `for` loop:

```
for (int i = 0; i + 1 < ARRAY_SIZE; i += 2)
```

The idea is that all iterations are always executed, independently of the input data. Instead of stopping the execution when the bound is crossed, the design simply stops updating the output value using the boolean variable `found`.

The variables `crossed1` and `crossed2` check whether the current sums exceed the bound, once one of the sums crosses the threshold, the variable `found` becomes true. After that point, the result is no longer updated, but the loop continues executing until completion. This preserves a static schedule while still producing the correct output.

The second optimization was manual loop unrolling by a factor of 2. Each iteration now processes two elements:

```
sum1 = a[i] + b[i];
sum2 = a[i + 1] + b[i + 1];
```

This increases the available parallelism because the two additions can execute simultaneously. The third optimization was loop pipelining: `#pragma HLS pipeline II=1 style=stp`. Since the two additions are independent and the design no longer contains a data dependent exit, a new iteration can start every cycle.

The fourth optimization was array partitioning for both the variable `a` and `b`. This partitioning is required because, with unroll factor 2, the design performs four parallel memory reads every cycle: two reads from `a` and two reads from `b`. I used cyclic partitioning because consecutive indices are accessed simultaneously: `a[i]`, `a[i+1]`, `b[i]`, `b[i+1]`. Each cyclic partition splits the array into 2 independent BRAMs, providing the 2 read ports per array per cycle that the unrolled body requires.

Finally, I added a final conditional check outside the loop:

```
if ((ARRAY_SIZE % 2) == 1 && !found)
```

This handles the case where the array size is odd and one element remains unprocessed after the factor-2 unrolling. Since this condition is outside the pipelined loop, it does not affect the static scheduling of the main datapath.

## Scheduling Dataflow Graph

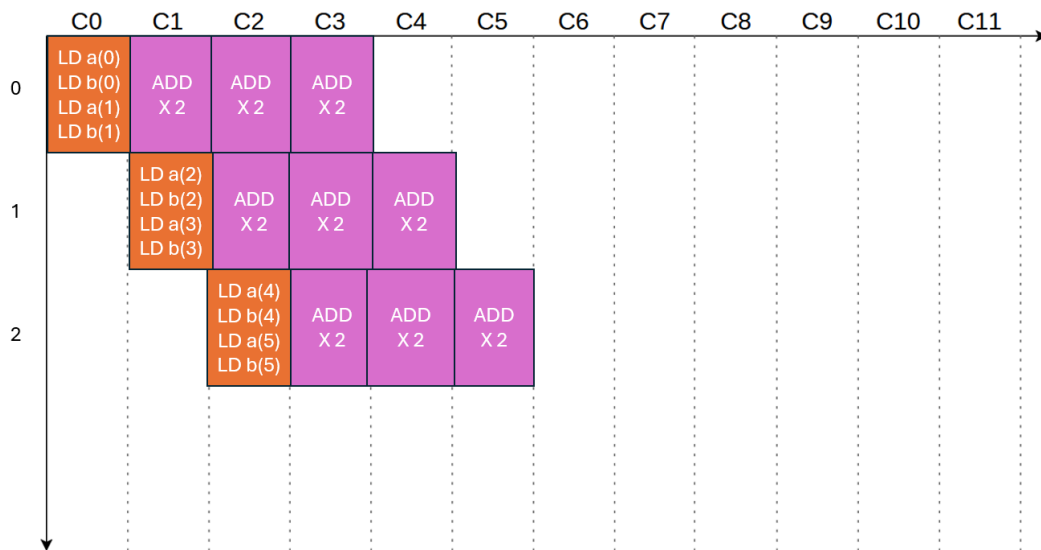


Figure 11: Scheduling dataflow graph for the first three iterations.

I modeled float comparisons like `sum >= bound` as contributing with zero cycles to the schedule, since in the HW they were not specified in the latency table.

## Cycle Count

| Quantity      | Number of cycles           |
|---------------|----------------------------|
| One iteration | 4                          |
| Entire task   | $\text{ARRAY\_SIZE}/2 + 3$ |

## Resources Required

| Resources Required |             |              |              |
|--------------------|-------------|--------------|--------------|
| Adders             | Multipliers | Ports (a)    | Ports (b)    |
| 2 (float)          | 0           | 2 read ports | 2 read ports |

## Trade-off

The optimized implementation removes the data-dependent loop exit and enables a fully pipelined static schedule with  $II = 1$ . Together with the manual unrolling by 2 transformation, this brings the total latency down to  $ARRAY\_SIZE/2 + 3$  cycles, against a baseline of roughly  $5 \times ARRAY\_SIZE$  cycles for the original `while` loop. In the worst case, the optimized design is therefore approximately 10x faster.

This speedup brings a hardware cost: the design now instantiates 2 float adders (against 1 in the baseline), 2 float comparators, additional select and OR logic for the `found` and `result` update, and requires cyclic partitioning of both `a` and `b` by factor 2, doubling the BRAM count for each array.

The limitation of this static optimization is that it always executes the full  $ARRAY\_SIZE/2$  iterations, even when the bound is exceeded very early

## Question 2.2 Dynamic HLS

As explained before, Kernel 5 has a runtime dependent exit condition. Therefore, in a Static HLS implementation, the scheduler worked in the worst case scenario and executed always the full loop.

A Dynamic HLS implementation could improve by directly stopping the iterations in the moment in which the bound gets crossed.

This can significantly improve the average performance, in particular in the case in which the bound gets crossed after a few iterations, executing only a small fraction of the loop iterations. Consider for example a data distribution with a relatively small bound and values of `a[i]` and `b[i]` large enough to exceed the bound after only a few iterations. For instance:

```
bound = 10
a = [4, 6, 1, 7, ...]
b = [8, 7, 3, 1, ...]
```

The first computed sum is:

$$4 + 8 = 12$$

therefore the bound is already exceeded during the first iterations. In a Dynamic HLS implementation, the computation could terminate almost immediately after detecting the first threshold crossing, executing only a small fraction of the loop iterations. The static optimization from Section 2.1, in contrast, would still execute all  $ARRAY\_SIZE/2 + 3$  iterations, because the datapath and schedule are fixed at compile time. On an array of size 1024, dynamic would finish in roughly 5 cycles while static would finish in 515 cycles, because the datapath and scheduling are fixed statically.